

重庆邮电大学

学生实验实习报告册

学年学期： 2025-2026 学年（1）学期

课程名称： 通信软件开发与应用

学生学院： 通信与信息工程学院

专业班级： 01042301

学生学号： 2023211281

学生姓名： 丁同勳

联系电话： 18019582857

重庆邮电大学教务处制

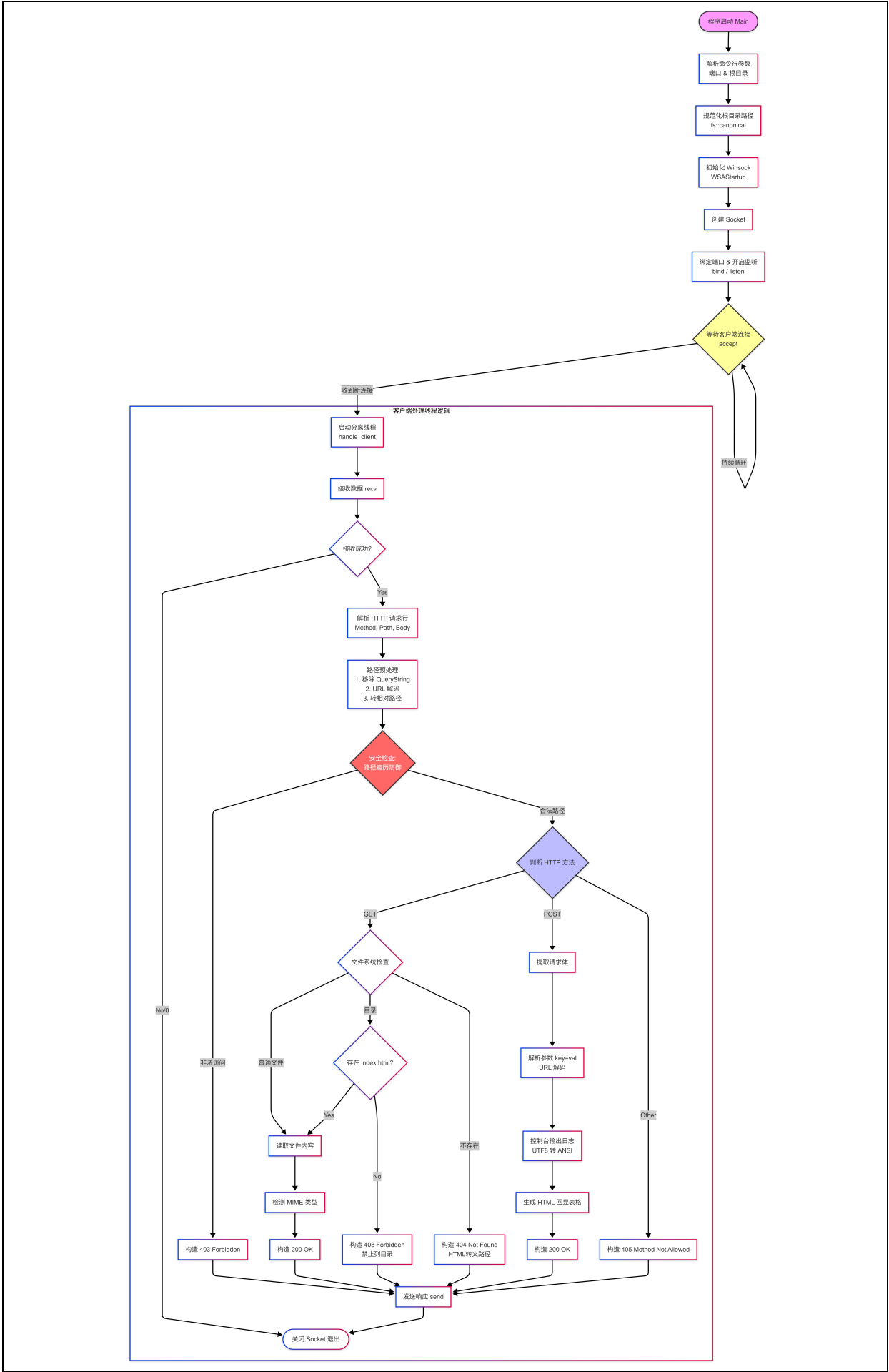
攻击。

○ **POST 请求:**

- 提取 HTTP Body 部分。
- 解析 key=value&key2=value2 格式的表单数据。
- **控制台输出:** 将解析的数据从 UTF-8 转换为 Windows ANSI 编码 (如 GBK), 确保在中文控制台正确打印。
- **浏览器响应:** 生成一个包含表格的 HTML 页面, 回显提交的数据, 返回 **200 OK**。

○ **其他方法:** 对于 PUT/DELETE 等未实现的方法, 返回 **405 Method Not Allowed**。

5. **发送与清理:** 调用 send 将构造好的 HTTP 响应发回客户端, 最后关闭 Socket 释放资源, 线程结束。



三、源代码

```
=====WebServer.cpp=====

/**
 * Simple C++ HTTP Server (Winsock)
 * 功能：支持静态文件服务（GET）和简单的表单回显（POST）
 * 特性：多线程处理、MIME类型检测、URL解码、路径遍历防御、中文控制台输出支持
 */

#define _WINSOCK_DEPRECATED_NO_WARNINGS

// =====
// Include Headers
// =====
#include <iostream>
#include <string>
#include <vector>
#include <thread>
#include <fstream>
#include <sstream>
#include <filesystem>
#include <map>
#include <iomanip>

// Windows Socket Headers
#include <winsock2.h>
#include <ws2tcpip.h>

// 链接 ws2_32.lib
#pragma comment(lib, "ws2_32.lib")

namespace fs = std::filesystem;

// =====
// Constants & Globals
// =====
const int DEFAULT_PORT = 9000;
const std::string DEFAULT_ROOT = ".";

// =====
// Helper Functions
// =====

/// <summary>
/// 根据文件后缀获取 MIME 类型
/// </summary>
```

```

std::string get_mime_type(const std::string& extension) {
    static const std::map<std::string, std::string> mime_types = {
        {".html", "text/html"},
        {".htm", "text/html"},
        {".css", "text/css"},
        {".js", "application/javascript"},
        {".txt", "text/plain"},
        {".jpg", "image/jpeg"},
        {".png", "image/png"}
    };
    auto it = mime_types.find(extension);
    if (it != mime_types.end()) return it->second;
    return "text/plain";
}

/// <summary>
/// 简单的 HTML 转义, 防止 XSS 或页面结构破坏
/// </summary>
std::string html_escape(const std::string& s) {
    std::string out;
    out.reserve(s.size());
    for (char c : s) {
        switch (c) {
            case '&': out += "&amp;"; break;
            case '<': out += "&lt;"; break;
            case '>': out += "&gt;"; break;
            case '"': out += "&quot;"; break;
            case '\': out += "&#39;"; break;
            default: out += c; break;
        }
    }
    return out;
}

/// <summary>
/// UTF-8 转 ANSI (用于 Windows 控制台正常显示中文)
/// </summary>
std::string Utf8ToAnsi(const std::string& utf8) {
    // 1. UTF-8 -> UTF-16 (Wide Char)
    int wlen = MultiByteToWideChar(CP_UTF8, 0, utf8.c_str(), -1, NULL, 0);
    if (wlen == 0) return "";
    std::vector<wchar_t> wbuf(wlen);
    MultiByteToWideChar(CP_UTF8, 0, utf8.c_str(), -1, wbuf.data(), wlen);

    // 2. UTF-16 -> ANSI (系统默认 Codepage, 中文环境通常是 GBK)
    int len = WideCharToMultiByte(CP_ACP, 0, wbuf.data(), -1, NULL, 0, NULL, NULL);

```

```

    if (len == 0) return "";
    std::vector<char> buf(len);
    WideCharToMultiByte(CP_ACP, 0, wbuf.data(), -1, buf.data(), len, NULL, NULL);

    return std::string(buf.data());
}

/// <summary>
/// URL 解码 (例如: %20 -> 空格)
/// </summary>
std::string url_decode(const std::string& encoded) {
    std::string decoded;
    decoded.reserve(encoded.length());
    for (size_t i = 0; i < encoded.length(); ++i) {
        if (encoded[i] == '%' && i + 2 < encoded.length()) {
            std::string hex = encoded.substr(i + 1, 2);
            char ch = static_cast<char>(std::strtol(hex.c_str(), nullptr, 16));
            decoded += ch;
            i += 2;
        }
        else if (encoded[i] == '+') {
            decoded += ' ';
        }
        else {
            decoded += encoded[i];
        }
    }
    return decoded;
}

/// <summary>
/// 解析 POST Body (格式: key1=val1&key2=val2)
/// </summary>
std::map<std::string, std::string> parse_post_body(const std::string& body) {
    std::map<std::string, std::string> params;
    std::stringstream ss(body);
    std::string pair;
    while (std::getline(ss, pair, '&')) {
        size_t pos = pair.find('=');
        if (pos != std::string::npos) {
            std::string key = url_decode(pair.substr(0, pos));
            std::string val = url_decode(pair.substr(pos + 1));
            params[key] = val;
        }
    }
    return params;
}

```

```

}

struct HttpRequest {
    std::string method;
    std::string path;
    std::string body;
};

// =====
// Core Logic
// =====

void handle_client(SOCKET clientSocket, std::string rootDir) {
    char buffer[8192];
    int bytesReceived = recv(clientSocket, buffer, 8192, 0);

    if (bytesReceived <= 0) {
        closesocket(clientSocket);
        return;
    }

    // 1. 解析请求行
    std::string rawRequest(buffer, bytesReceived);
    HttpRequest request;
    std::istringstream stream(rawRequest);
    std::string line;

    if (std::getline(stream, line)) {
        std::istringstream lineStream(line);
        lineStream >> request.method >> request.path;
    }

    // 2. 预处理路径
    // 移除 query string (?后面的内容)
    size_t qpos = request.path.find('?');
    if (qpos != std::string::npos) request.path = request.path.substr(0, qpos);

    // URL 解码
    std::string decodedPath = url_decode(request.path);

    // 去掉起始的 '/' 或 '\\', 转换为相对路径
    while (!decodedPath.empty() && (decodedPath[0] == '/' || decodedPath[0] == '\\')) {
        decodedPath.erase(0, 1);
    }

    if (decodedPath.empty()) decodedPath = "index.html";
}

```



```

// 打印请求日志
std::cout << "[Thread " << std::this_thread::get_id() << "]" << " "
    << request.method << " request for: " << request.path << std::endl;

std::string response;

try {
    fs::path rootPath(rootDir);
    fs::path targetPath = fs::weakly_canonical(rootPath / decodedPath);

    // 3. 安全检查: 防止路径遍历攻击 (Path Traversal)
    // 确保最终解析的路径必须以 rootPath 开头
    std::string rootStr = fs::canonical(rootPath).string();
    std::string targetStr = targetPath.string();

    if (targetStr.size() < rootStr.size() || targetStr.compare(0, rootStr.size(), rootStr) !=
0) {
        // 访问 root 之外的路径 -> 403 Forbidden
        std::string bodyContent = "<html><head><meta charset='utf-8'></head><body><h1>403
Forbidden</h1><p>Access denied.</p></body></html>";
        response = "HTTP/1.1 403 Forbidden\r\n"
            "Content-Type: text/html; charset=utf-8\r\n"
            "Content-Length: " + std::to_string(bodyContent.size()) + "\r\n\r\n" +
            bodyContent;

        send(clientSocket, response.c_str(), (int)response.size(), 0);
        closesocket(clientSocket);
        return;
    }

    // 4. 处理 GET 请求
    if (request.method == "GET") {
        if (fs::exists(targetPath) && fs::is_regular_file(targetPath)) {
            // 文件存在且是普通文件 -> 返回文件内容
            std::ifstream file(targetPath, std::ios::binary);
            if (file) {
                std::stringstream buffer;
                buffer << file.rdbuf();
                std::string content = buffer.str();
                std::string mime = get_mime_type(fs::path(targetPath).extension().string());

                response = "HTTP/1.1 200 OK\r\n"
                    "Content-Type: " + mime + "\r\n"
                    "Content-Length: " + std::to_string(content.size()) + "\r\n\r\n" +
                    content;
            }
        }
    }
}

```

```

    }
    else {
        response = "HTTP/1.1 500 Internal Server Error\r\n\r\n";
    }
}

else if (fs::exists(targetPath) && fs::is_directory(targetPath)) {
    // 如果是目录, 尝试查找 index.html
    fs::path indexPath = targetPath / "index.html";
    if (fs::exists(indexPath) && fs::is_regular_file(indexPath)) {
        std::ifstream file(indexPath, std::ios::binary);
        if (file) {
            std::stringstream buffer;
            buffer << file.rdbuf();
            std::string content = buffer.str();
            std::string mime =
get_mime_type(fs::path(indexPath).extension().string());

            response = "HTTP/1.1 200 OK\r\n"
                "Content-Type: " + mime + "\r\n"
                "Content-Length: " + std::to_string(content.size()) + "\r\n\r\n" +
                content;
        }
        else {
            response = "HTTP/1.1 500 Internal Server Error\r\n\r\n";
        }
    }
    else {
        // 目录存在但没有 index -> 返回 403 (禁止列目录)
        std::string bodyContent = "<html><head><meta
charset='utf-8'></head><body><h1>403 Forbidden</h1><p>Directory access is
forbidden.</p></body></html>";
        response = "HTTP/1.1 403 Forbidden\r\nContent-Type: text/html;
charset=utf-8\r\nContent-Length: " + std::to_string(bodyContent.size()) + "\r\n\r\n" +
bodyContent;
    }
}

else {
    // 资源不存在 -> 返回自定义 404 页面
    std::string escaped = html_escape(request.path);
    std::string bodyContent = "<html><head><meta charset='utf-8'></head><body><h1>404
Not Found</h1><p>Requested resource: " + escaped + " not found.</p></body></html>";
    response = "HTTP/1.1 404 Not Found\r\nContent-Type: text/html;
charset=utf-8\r\nContent-Length: " + std::to_string(bodyContent.size()) + "\r\n\r\n" +
bodyContent;
}
}
}

```

```

// 5. 处理 POST 请求
else if (request.method == "POST") {
    // 提取 Body
    size_t bodyPos = rawRequest.find("\r\n\r\n");
    if (bodyPos != std::string::npos) {
        request.body = rawRequest.substr(bodyPos + 4);
    }

    // 解析参数 (此时 params 里的 string 还是 UTF-8 编码)
    auto params = parse_post_body(request.body);

    // --- 构建响应给浏览器 (保持 UTF-8) ---
    std::stringstream html;
    html << "<html><head><meta charset='utf-8'></head><body>"; // 确保浏览器知道是UTF-8
    html << "<h1>POST Received</h1>";
    html << "<table border='1'>";
    for (const auto& kv : params) {
        html << "<tr><td>" << kv.first << "</td><td>" << kv.second << "</td></tr>";
    }
    html << "</table></body></html>";
    std::string bodyContent = html.str();

    response = "HTTP/1.1 200 OK\r\n";
    response += "Content-Type: text/html; charset=utf-8\r\n";
    response += "Content-Length: " + std::to_string(bodyContent.size()) + "\r\n\r\n";
    response += bodyContent;

    // --- 打印到控制台 (转为 ANSI/GBK 以避免乱码) ---
    std::cout << " [POST Data Decoded]:" << std::endl;
    for (const auto& kv : params) {
        std::cout << "      " << Utf8ToAnsi(kv.first) << " = " << Utf8ToAnsi(kv.second) <<
std::endl;
    }
}

else {
    response = "HTTP/1.1 405 Method Not Allowed\r\n\r\n";
}

}

catch (const std::exception& e) {
    // 任何文件系统异常都作为 403/500 处理, 避免泄露内部路径信息
    response = "HTTP/1.1 500 Internal Server Error\r\nContent-Length: 0\r\n\r\n";
}

send(clientSocket, response.c_str(), (int)response.size(), 0);
closesocket(clientSocket);
}

```

```

// =====
// Main Entry Point
// =====
int main(int argc, char* argv[]) {
    int port = DEFAULT_PORT;
    std::string rootDir = DEFAULT_ROOT;

    // 解析命令行参数
    if (argc > 1) port = std::stoi(argv[1]);
    if (argc > 2) rootDir = argv[2];

    // 规范化根目录为绝对并 canonical，方便后续比较和防止路径遍历
    try {
        fs::path rp = fs::absolute(rootDir);
        rp = fs::canonical(rp);
        rootDir = rp.string();
    }
    catch (...) {
        // 如果 canonical 失败（目录不存在），转为 absolute 但不要抛出异常，保持程序运行
        rootDir = fs::absolute(rootDir).string();
    }

    // 初始化 Winsock
    WSADATA wsaData;
    if (WSAStartup(MAKEWORD(2, 2), &wsaData) != 0) {
        std::cerr << "WSAStartup failed." << std::endl;
        return 1;
    }

    // 创建 Socket
    SOCKET serverSocket = socket(AF_INET, SOCK_STREAM, 0);
    if (serverSocket == INVALID_SOCKET) {
        std::cerr << "Socket creation failed." << std::endl;
        WSACleanup();
        return 1;
    }

    sockaddr_in serverAddr;
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(port);

    // 绑定 & 监听
    if (bind(serverSocket, (sockaddr*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) {
        std::cerr << "Bind failed." << std::endl;
    }
}

```

```

        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    if (listen(serverSocket, SOMAXCONN) == SOCKET_ERROR) {
        std::cerr << "Listen failed." << std::endl;
        closesocket(serverSocket);
        WSACleanup();
        return 1;
    }

    std::cout << "Server running on port " << port << std::endl;
    std::cout << "Serving files from: " << fs::absolute(rootDir).string() << std::endl;
    std::cout << "Test URL: http://localhost:" << port << "/" << std::endl;

    // 接受连接循环
    while (true) {
        SOCKET clientSocket = accept(serverSocket, nullptr, nullptr);
        if (clientSocket != INVALID_SOCKET) {
            // 为每个客户端启动一个新线程
            std::thread(handle_client, clientSocket, rootDir).detach();
        }
    }

    // 清理资源 (虽然死循环不会运行到这里)
    closesocket(serverSocket);
    WSACleanup();
    return 0;
}

```

=====index.html=====

```

<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>C++ WebServer 测试页</title>
    <style>
        body {
            font-family: "Microsoft YaHei", Arial, sans-serif;
            max-width: 800px;
            margin: 0 auto;
            padding: 20px;
            background-color: #f4f4f9;
            color: #333;
        }
    
```

```
h1 {
  color: #2c3e50;
  border-bottom: 2px solid #3498db;
  padding-bottom: 10px;
}

h2 {
  color: #2980b9;
  margin-top: 30px;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  margin-bottom: 20px;
}

.btn {
  background-color: #27ae60;
  color: white;
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  font-size: 16px;
  white-space: nowrap; /* 防止按钮文字换行 */
}

.btn:hover {
  background-color: #219150;
}

/* 输入框容器样式，让输入框和按钮并排 */
.input-group {
  display: flex;
  gap: 10px;
  align-items: center;
  margin-bottom: 15px;
}

input[type="text"], textarea {
  width: 100%;
  padding: 10px;
```

```

        margin: 5px 0;
        box-sizing: border-box;
        border: 1px solid #ccc;
        border-radius: 4px;
    }

    /* 特定调整让并排输入框自适应宽度 */
    .input-group input[type="text"] {
        margin: 0;
        flex-grow: 1;
    }

    a {
        color: #3498db;
        text-decoration: none;
    }

    a:hover {
        text-decoration: underline;
    }

    .quick-links {
        font-size: 0.9em;
        color: #666;
        margin-top: 10px;
    }
</style>
</head>
<body>

<h1>C++ WebServer 运行成功</h1>
<p>如果页面没有乱码，说明服务器的 HTML 解析和 UTF-8 编码处理正常。</p>

<!-- 测试 1: 自定义 GET 请求 -->
<div class="card">
    <h2>📁 测试 1: GET 请求（指定文件读取）</h2>
    <p>输入你想读取的文件名（包含扩展名），点击按钮向服务器发送请求。</p>

    <div class="input-group">
        <input type="text" id="fileNameInput" placeholder="例如: test.txt, index2.html, data.json" value="test.txt">
        <button class="btn" onclick="requestFile()">读取文件</button>
    </div>

    <div class="quick-links">
        <strong>快速测试链接: </strong>

```

```

        <a href="javascript:void(0)" onclick="setAndGo(' test.txt')">test.txt</a> |
        <a href="javascript:void(0)" onclick="setAndGo(' index2.html')">index2.html</a> |
        <a href="javascript:void(0)" onclick="setAndGo(' image.jpg')">image.jpg</a> |
        <a href="/non_existent_file.html" target="_blank">测试 404 (文件不存在)</a>

    </div>
</div>

<!-- 测试 2: POST 请求 -->
<div class="card">
    <h2>📄 测试 2: POST 请求 (提交表单)</h2>
    <p>在下方输入内容并提交，服务器将接收数据并回显 (Echo) 。</p>
    <form action="/submit_data" method="POST">
        <label for="username">姓名 (Name):</label>
        <input type="text" id="username" name="username" value="张三" placeholder="输入你的名字">

        <label for="message">消息 (Message):</label>
        <textarea id="message" name="message" rows="3">Hello C++ Server!</textarea>

        <input type="submit" value="发送 POST 请求" class="btn" style="width: 100%;
margin-top: 10px;">
    </form>
</div>

<div style="text-align: center; color: #777; font-size: 0.9em;">
    Powered by Native C++ Winsock
</div>

<script>
    // 处理文件请求跳转
    function requestFile() {
        var fileName = document.getElementById(' fileNameInput').value.trim();

        if (!fileName) {
            alert("请输入要读取的文件名!");
            return;
        }

        // 如果用户没有输入开头的 /，虽然浏览器通常会自动处理，
        // 但为了清晰，直接用相对路径打开即可
        // _blank 表示在新标签页打开，方便对比
        window.open(fileName, '_blank');
    }

    // 快速链接辅助函数
    function setAndGo(file) {

```



```
        document.getElementById('fileNameInput').value = file;
        requestFile();
    }
</script>

</body>
</html>
```

四、实验结果及分析

构建的简单测试页面

C++ WebServer 运行成功

如果页面没有乱码，说明服务器的 HTML 解析和 UTF-8 编码处理正常。

📁 测试 1: GET 请求 (指定文件读取)

输入你想读取的文件名（包含扩展名），点击按钮向服务器发送请求。

读取文件

快速测试链接: [test.txt](#) | [index2.html](#) | [image.jpg](#) | [测试 404 \(文件不存在\)](#)

📄 测试 2: POST 请求 (提交表单)

在下方输入内容并提交，服务器将接收数据并回显 (Echo)。

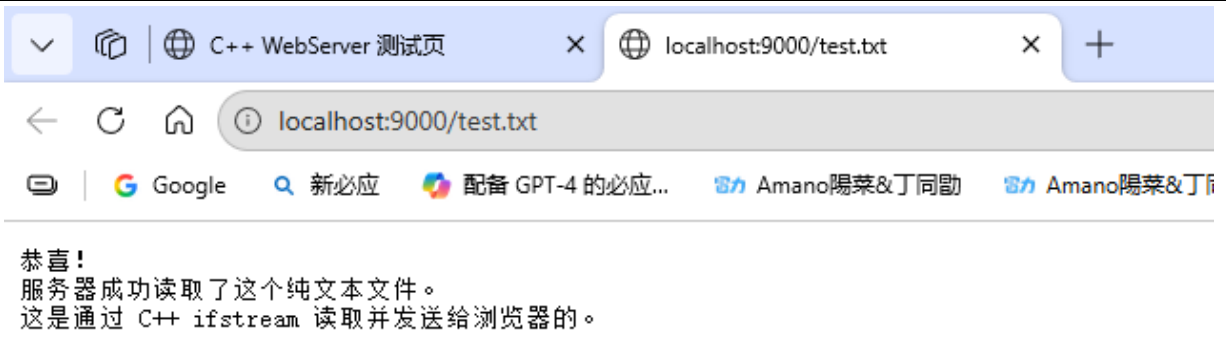
姓名 (Name):

消息 (Message):

发送 POST 请求

Powered by Native C++ Winsock

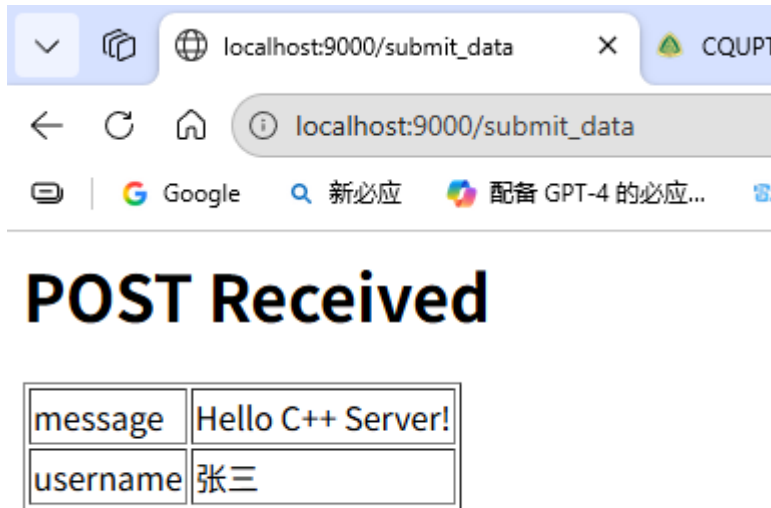
GET 根目录 .txt 文件



GET 根目录 .html 文件



POST 表单



Server 日志页面

```
DA\个人文件\作业5-第五学期 x + v
Server running on port 9000
Serving files from: D:\个人文件\作业5-第五学期_大三上\通信软件开发与应用\Development_and_Application_of_Communication_Software\Task_6_WebServer
Test URL: http://localhost:9000/
[Thread 37092] GET request for: /
[Thread 36948] GET request for: /hybridaction/zybTrackerStatisticsAction
[Thread 16144] GET request for: /hybridaction/zybTrackerStatisticsAction
[Thread 37584] GET request for: /test.txt
[Thread 35976] GET request for: /hybridaction/zybTrackerStatisticsAction
[Thread 36776] GET request for: /hybridaction/zybTrackerStatisticsAction
[Thread 25748] GET request for: /index2.html
[Thread 13388] GET request for: /hybridaction/zybTrackerStatisticsAction
[Thread 23592] GET request for: /hybridaction/zybTrackerStatisticsAction
[Thread 32156] POST request for: /submit_data
[POST Data Decoded]:
  message = Hello C++ Server!
  username = 张三
[Thread 31712] GET request for: /hybridaction/zybTrackerStatisticsAction
[Thread 35880] GET request for: /hybridaction/zybTrackerStatisticsAction
```

五、实验中问题及分析

1. 网络连接与端口问题

问题 A: 程序启动报错 "Bind failed"

- 现象: 运行程序后, 控制台直接输出 "Bind failed" 并退出。
- 分析:
 1. 端口冲突: 代码默认使用 9000 端口, 该端口可能已经被其他程序 (如 PHP-FPM, Docker, 其他 Web 服务) 占用。
 2. 权限不足: 虽然 9000 不是知名端口 (<1024), 但在某些严格的安全策略下可能被拦截。
- 解决办法:
 - 检查端口: 在 CMD 中运行 `netstat -ano | findstr 9000` 查看是否有进程占用。
 - 更换端口: 通过命令行参数启动, 例如 `server.exe 8080`, 或修改代码中的 `DEFAULT_PORT` 常量。

问题 B: 浏览器无法访问 (一直在加载或连接超时)

- 现象: 服务器显示 "Running", 但浏览器访问 `localhost:9000` 无响应, 或者局域网其他电脑无法访问。
- 分析: **Windows 防火墙** 拦截了入站连接。特别是在局域网测试时, 默认防火墙规则会阻止非标准端口的外部访问。
- 解决办法:
 - 本地测试: 确保访问的是 `127.0.0.1` 或 `localhost`。
 - 局域网测试: 打开 Windows 防火墙 -> 高级设置 -> 入站规则 -> 新建规则 -> 端口 -> TCP -> 特定本地端口 9000 -> 允许连接。

2. 中文乱码问题

问题 C: 控制台输出的 POST 数据中文乱码

- 现象: 浏览器提交表单 "姓名=张三", 服务器控制台显示 "姓名=ç§â..." 或其他乱码。
- 分析:
 - 现代浏览器提交表单默认使用 **UTF-8** 编码。
 - Windows 中文控制台 (CMD/PowerShell) 默认代码页通常是 **GBK (CP936)**。

- **解决办法:**
 - 代码中已经包含了 `Utf8ToAnsi` 函数试图解决此问题。
 - 或者, 移除代码中的转码逻辑, 并在 CMD 中输入 `chcp 65001` 切换为 UTF-8 模式运行程序。

问题 D: 浏览器显示的静态 HTML 中文乱码

- **现象:** 访问 `index.html`, 浏览器里中文显示乱码。
- **分析:**
 1. **文件保存编码:** `index.html` 文件保存时是 GBK 编码 (记事本默认), 但 HTTP 响应头写的是 `Content-Type: text/html; charset=utf-8`。
 2. **响应头缺失:** 如果响应头没指明 `charset`, 浏览器会瞎猜。
- **解决办法:**
 - **统一编码:** 使用 VS Code 或 Notepad++ 将所有 HTML/TXT 文件显式保存为 **UTF-8 (无 BOM)** 格式。
 - 代码中强制了 `charset=utf-8`, 所以必须保证源文件也是 UTF-8。

3. 程序逻辑与稳定性问题

问题 E: 读取大文件导致程序崩溃 (内存溢出)

- **现象:** 尝试下载一个大文件 (例如 1GB 的视频) 时, 服务器直接崩溃或占用极大内存。
- **分析:** 代码逻辑使用了 `std::stringstream` 一次性读取整个文件内容:

```
buffer << file.rdbuf(); // 危险操作
```

这会将整个文件加载到 RAM 中。如果文件大于可用内存, 程序会抛出 `std::bad_alloc` 异常或被系统杀掉。

- **解决办法 (进阶修改):**
 - 不要一次性读取。应该循环读取固定大小的块 (例如 8KB), 并分块发送 (`send`)。
 - 修改 `handle_client` 中的发送逻辑, 改为 `while(file.read(buf, sizeof(buf))) { send(...); }`。

问题 F: 安全隐患 (路径遍历)

- **现象:** 攻击者可能用路径拼接访问 `http://localhost:9000/../../Windows/win.ini`。
- **分析:** 代码使用了 `fs::weakly_canonical` 和前缀比对。
- **实验验证:** 为了验证安全性, 可以尝试在 URL 输入 `../` 看是否被拦截。

六、心得体会

1. 亲身体会 HTTP 协议

在接触 High-level 框架 (如 Java SpringBoot 或 Python Flask) 之前, HTTP 对我来说只是一个抽象的概念。通过亲手解析原始的 `char` 缓冲数据, 我深刻认识到 **HTTP 的本质就是基于 TCP 连接的文本协议**。

- **体会:** 服务器只需要按照固定的格式 (请求行、Header、空行、Body) 去解析字符串, 并按照标准格式构造响应字符串。例如, 漏掉一个 `\r\n` 或者 `Content-Length` 计算错误, 浏览器就无法正确渲染, 这让我对协议规范的严谨性有了直观的认识。

2. 多线程并发模型的直观理解

在单线程版本中, 如果一个客户端正在传输大文件, 其他客户端只能阻塞等待, 体验极差。

- **体会:** 实验中引入 `std::thread` 并将 `clientSocket` 移交给子线程处理, 实现了简单的并发服务

器。我观察到多个浏览器窗口可以同时加载页面，这让我理解了**阻塞式 I/O (Blocking I/O)** 在主线程中的局限性，以及多线程如何通过“空间换时间”来提升服务器吞吐量。同时也意识到，虽然多线程解决了并发问题，但创建线程的开销较大，未来可以尝试使用**线程池**或**I/O 多路复用 (Select/Epoll/IOCP)** 来进一步优化。

3. 网络安全意识

以前写代码很少考虑安全问题，认为“功能实现”即结束。但在本实验中，**路径遍历攻击**的防御让我印象深刻。

- **体会：**如果没有 `std::filesystem::canonical` 和路径前缀比对，攻击者只需请求 `GET ../../windows/win.ini` 就能窃取服务器敏感文件。这让我明白，**永远不要信任客户端的输入**。所有的输入（包括路径、URL 参数、Header）都必须经过严格的清洗和验证，安全校验必须在业务逻辑执行之前完成。

4. 编码与环境差异的“坑”

实验中花费时间最多的地方不是逻辑，而是**字符编码 (Encoding)** 问题。

- **体会：**浏览器的 UTF-8 与 Windows 控制台的 GBK (ANSI) 之间的冲突让人头疼。通过编写 `Utf8ToAnsi` 函数，我深入理解了多字节字符集与宽字符集（Wide Char）的区别。这提醒我，在开发跨平台或网络应用时，必须统一使用 UTF-8 标准，并在边界处小心处理编码转换，否则乱码问题将无处不在。

